

APL 405: Machine Learning

Lecture 5: Neural Network

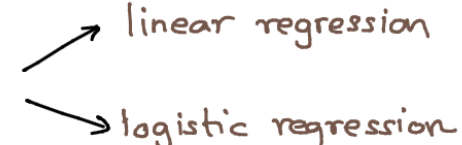
by

Sabyasachi Chatterjee

Assistant Professor,
Applied Mechanics Department,
IIT Delhi

Instructor email: sabyasachi@am.iitd.ac.in

Neural Networks

- We already looked at two basic parametric models 
 - linear regression
 - logistic regression
- A **neural network**, in some sense, extends these basic models by stacking multiple copies of these models to construct a **hierarchical model**
- This hierarchical model can describe **more complicated relationships** between inputs and outputs than a linear or logistic regression
- **Deep learning** is a subfield of machine learning that deals with such hierarchical machine learning models

Neural Network Model

- Earlier we introduced the concept of **non-linear parametric functions** for modelling the relationship between input variables $x_1, \dots, x_p$ and output y

$$\hat{y} = f_{\underline{\theta}}(\underline{x}) = f_{\underline{\theta}}(x_1, x_2, \dots, x_p)$$

 the function f is "parameterized" by $\underline{\theta}$

- Such a non-linear function $f_{\underline{\theta}}(\cdot)$ can be created in many ways
- In neural network, the strategy is to use several **layers** of linear regression models and non-linear **activation functions**

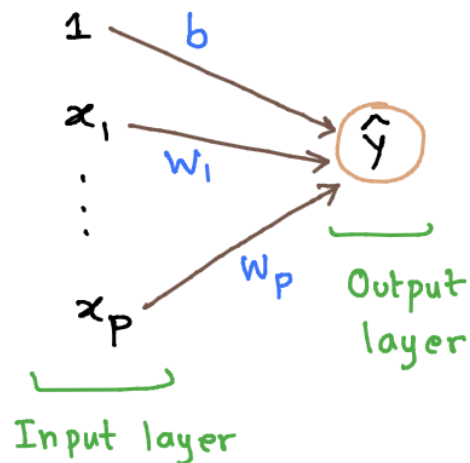
Neural Network model – linear regression

- We start the description of a neural network model with linear regression model

$$\hat{y} = \overset{\Theta_0}{b} + \overset{\Theta_1}{w_1}x_1 + \overset{\Theta_2}{w_2}x_2 + \dots + \overset{\Theta_p}{w_p}x_p$$

- $w_1, w_2, \dots, w_p$ are called the **weights** and b is the **offset**
- we use this notation because it is more popular in neural networks

Graphical representation

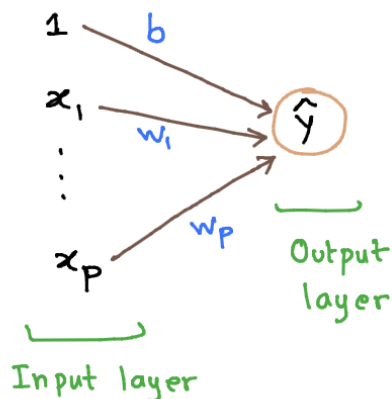


$$\hat{y} = w_1 x_1 + \dots + w_p x_p + b$$

$\xrightarrow{w_j}$ - Each link is associated with a parameter

$\hat{y}$ - Output $\hat{y}$ is described as sum of all terms

Neural network model – nonlinear relationship



$$\hat{y} = w_1 x_1 + \dots + w_p x_p + b$$

Describes a linear relationship

– How to describe a non-linear relationship between

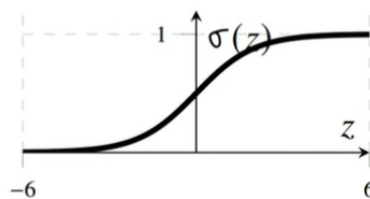
$$\underline{x} = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_p \end{bmatrix} \quad \text{and} \quad \hat{y} ?$$

– Use activation function $h: \mathbb{R} \rightarrow \mathbb{R}$

$$\hat{y} = \sigma(w_1 x_1 + \dots + w_p x_p + b)$$

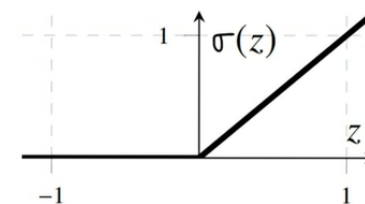
– Common choices of activation functions are:

Logistic: $\sigma(z) = \frac{1}{1 + e^{-z}}$
(Sigmoid)



$$\text{Logistic: } \sigma(z) = \frac{1}{1 + e^{-z}}$$

ReLU: $\sigma(z) = \max(0, z)$
(standard choice)



$$\text{ReLU: } \sigma(z) = \max(0, z)$$

Neural network – contd.

$$\hat{y} = \sigma(w_1 x_1 + \dots + w_p x_p + b)$$

This now becomes a
generalized linear model

- This generalized linear model is very simple
- Cannot describe very complicated relationships between input $\underline{x}$ and output $\hat{y}$

– How can we extend this simple model to increase the flexibility?

Stack several generalized linear models in a sequential construction

Ex:

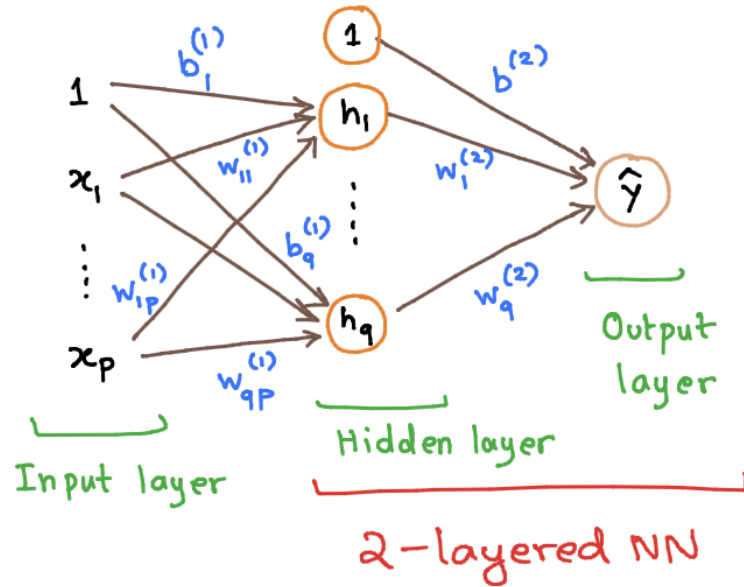
One-layer of stacking (hidden layer)

$$\begin{cases} h_1 = \sigma(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + \dots + w_{1p}^{(1)} x_p + b_1^{(1)}) \\ h_2 = \sigma(w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + \dots + w_{2p}^{(1)} x_p + b_2^{(1)}) \\ \vdots \\ h_q = \sigma(w_{q1}^{(1)} x_1 + w_{q2}^{(1)} x_2 + \dots + w_{qp}^{(1)} x_p + b_q^{(1)}) \end{cases}$$

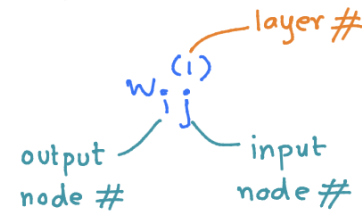
Output layer

$$\hat{y} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + \dots + w_q^{(2)} h_q + b^{(2)}$$

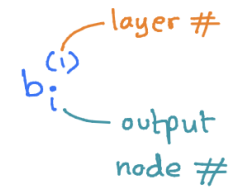
Two layered Neural Network



- Each link (arrow) is associated with a weight



- Each circular node is associated with its own bias term b



$$h_1 = \sigma(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + \dots + w_{1p}^{(1)} x_p + b_1^{(1)})$$

$$h_2 = \sigma(w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + \dots + w_{2p}^{(1)} x_p + b_2^{(1)})$$

$\vdots$

$$h_q = \sigma(w_{q1}^{(1)} x_1 + w_{q2}^{(1)} x_2 + \dots + w_{qp}^{(1)} x_p + b_q^{(1)})$$

$$\hat{y} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + \dots + w_q^{(2)} h_q + b^{(2)}$$

Two layered Neural Network – Vectorized representation

- The 2-layered neural network can be more compactly written using matrix notation:

$$\underline{\underline{W}}^{(1)} = \begin{bmatrix} w_{11}^{(1)} & \dots & w_{1p}^{(1)} \\ \vdots & & \vdots \\ w_{q1}^{(1)} & \dots & w_{qp}^{(1)} \end{bmatrix}, \quad q \times p$$

$$\underline{b}^{(1)} = \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_q^{(1)} \end{bmatrix}, \quad q \times 1$$

$$\underline{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{bmatrix}, \quad p \times 1$$

$$\underline{\underline{W}}^{(2)} = \begin{bmatrix} w_1^{(2)} & \dots & w_q^{(2)} \end{bmatrix}, \quad 1 \times q$$

$$\underline{b}^{(2)} = \begin{bmatrix} b^{(2)} \end{bmatrix}, \quad 1 \times 1$$

$$\underline{h} = \begin{bmatrix} h_1 \\ \vdots \\ h_q \end{bmatrix}, \quad q \times 1$$

Compact Representation

$$\underline{h} = \sigma \left(\underline{\underline{W}}^{(1)} \underline{x} + \underline{b}^{(1)} \right)$$

$q \times 1$ $q \times 1$ $p \times 1$

$$\hat{y} = \underline{\underline{W}}^{(2)} \underline{h} + \underline{b}^{(2)}$$

1×1 $1 \times q$ $q \times 1$ 1×1

Parameters

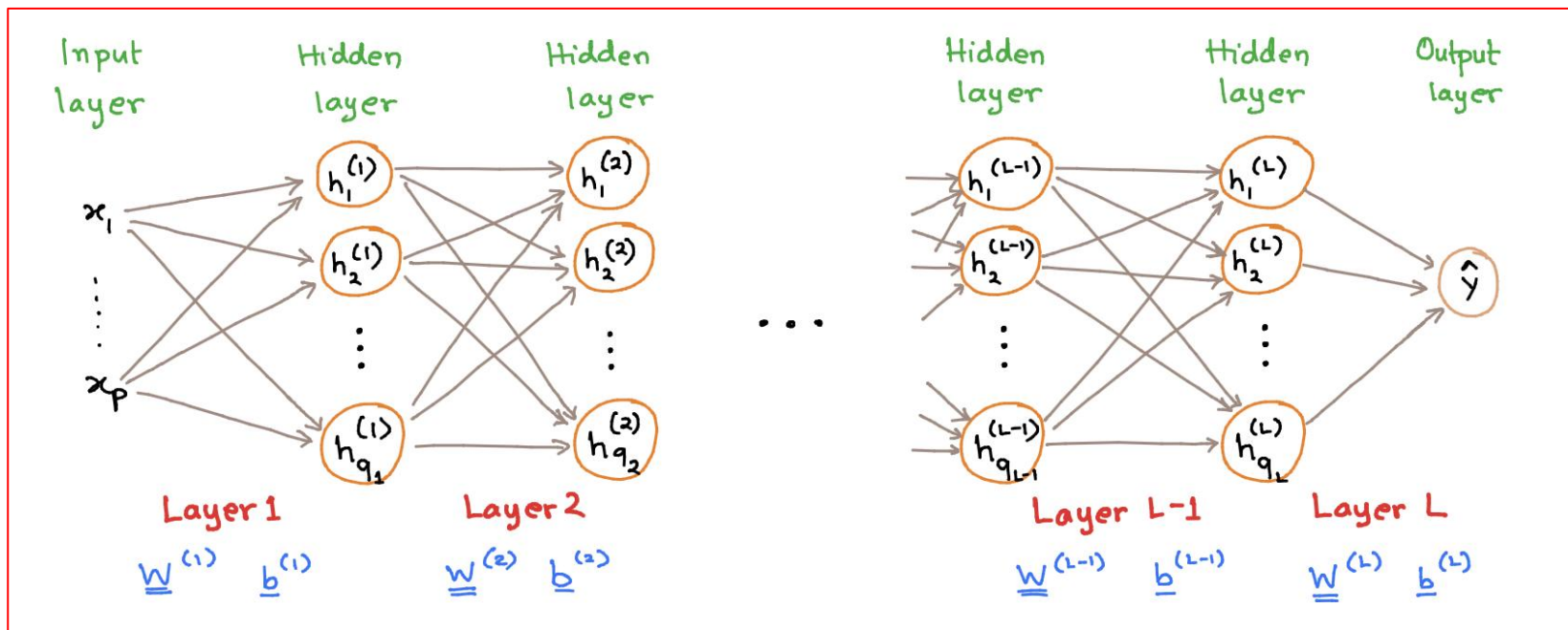
$$\underline{\Theta} = \begin{bmatrix} \text{vec}(\underline{\underline{W}}^{(1)}) \\ \underline{b}^{(1)} \\ \text{vec}(\underline{\underline{W}}^{(2)}) \\ b^{(2)} \end{bmatrix}$$

Regression problem

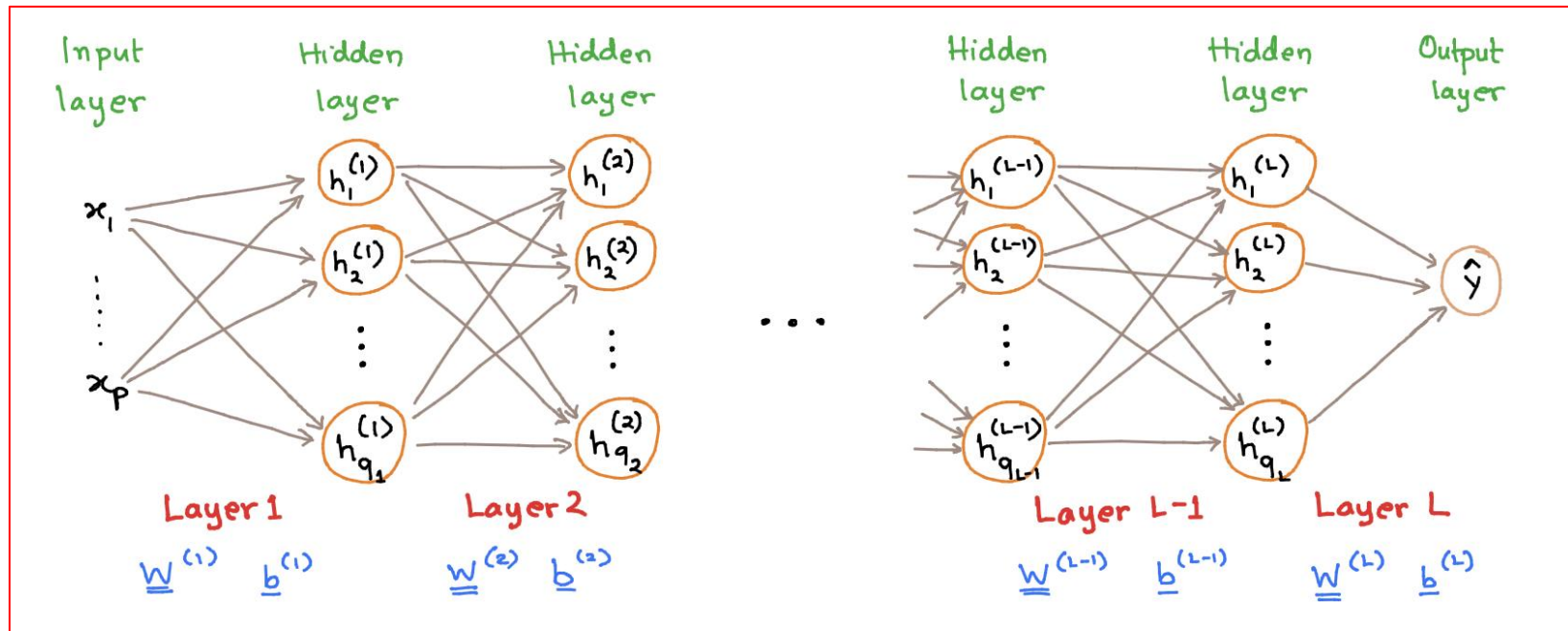
$$\hat{y} = f_{\underline{\Theta}}(\underline{x})$$

Deep Neural Network

- The 2-layered neural network is called **shallow NN** (because it has one hidden layer)
- The real flexibility of neural network comes when we have more than one hidden layer.
- Stacking multiple hidden layers leads to a **DEEP** neural network



Deep Neural Network (Feed-Forward Neural Network - FNN)



Mathematical
representation
of FNN:

$$\underline{h}^{(1)} = \sigma(\underline{\underline{W}}^{(1)} \underline{x} + \underline{b}^{(1)})$$

$$\underline{h}^{(2)} = \sigma(\underline{\underline{W}}^{(2)} \underline{h}^{(1)} + \underline{b}^{(2)})$$

$$\vdots$$

$$\underline{h}^{(L)} = \sigma(\underline{\underline{W}}^{(L-1)} \underline{h}^{(L-1)} + \underline{b}^{(L-1)})$$

$$\hat{y} = \underline{\underline{W}}^{(L)} \underline{h}^{(L-1)} + \underline{b}^{(L)}$$

Vectorization over data-points

- During training, the neural network model is used to compute the predicted output for several input points $\{y_i, x_i\}_{i=1}^N$

- The 2-layer neural network

column vector

$$\underline{h} = \sigma(\underline{W}^{(1)} \underline{x} + \underline{b}^{(1)})$$

$q \times 1$

$$\hat{y} = \underline{W}^{(2)} \underline{h} + \underline{b}^{(2)}$$

1×1

row vector

$$\underline{h}_i^T = \sigma(\underline{x}_i^T \underline{W}^{(1)T} + \underline{b}^{(1)T})$$

$1 \times q$

$$\hat{y}_i = \underline{h}_i^T \underline{W}^{(2)T} + \underline{b}^{(2)T}$$

1×1

$i = 1, 2, \dots, n$

Holds for each data point

Transposed

Stack all data points in matrices

$$\underline{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}_{N \times 1}, \quad \underline{X} = \begin{bmatrix} x_1^T \\ \vdots \\ x_N^T \end{bmatrix}_{N \times p}, \quad \hat{\underline{y}} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_N \end{bmatrix}_{N \times 1}, \quad \underline{H} = \begin{bmatrix} h_1^T \\ \vdots \\ h_N^T \end{bmatrix}_{N \times q}$$

each row represents a training data point

$$\underline{H} = \sigma(\underline{X} \underline{W}^{(1)T} + \underline{b}^{(1)T})$$

$$\hat{\underline{y}} = \underline{H} \underline{W}^{(2)T} + \underline{b}^{(2)T}$$

To be implemented in PyTorch, Tensorflow.

Neural Network for classification

- How did we extend linear regression to logistic regression?
 - By applying logistic (or sigmoid) function to the output of linear regression in case of binary classification
 - For multi-class classification (with $y \in \{1, 2, \dots, M\}$ classes), we used the softmax function

$$\text{softmax}(\underline{\underline{z}}) = \frac{1}{\sum_{j=1}^M e^{z_j}} \begin{bmatrix} e^{z_1} \\ e^{z_2} \\ \vdots \\ e^{z_M} \end{bmatrix}$$

- The softmax function now becomes an **additional** activation function, acting on the final layer of the neural network

$$\begin{aligned} \underline{h}^{(1)} &= \sigma(\underline{W}^{(1)} \underline{x} + \underline{b}^{(1)}) \\ &\vdots \\ \underline{h}^{(L-1)} &= \sigma(\underline{W}^{(L-1)} \underline{h}^{(L-2)} + \underline{b}^{(L-1)}) \\ \underline{\underline{z}}_{M \times 1} &= \underline{W}^{(L)} \underline{h}^{(L-1)} + \underline{b}^{(L)} \\ \underline{\underline{q}}_{M \times 1} &= \text{softmax}(\underline{\underline{z}}) \end{aligned}$$

Classification example

- Dataset has 60000 training images
- " " 10000 test/validation images
- Each data point consists of a 28x28 pixel grayscale image of a handwritten digit
- Each image is also labelled with the digit 0, 1, 2, ..., 9 that it depicts
- Each pixel intensity has been normalized between [0, 1]



- Consider image as input $\underline{x} = [x_1 \ x_2 \ \dots \ x_p]^T$
 - $p = 28 \times 28 = 784$ input variables (flattened out)
 - Each x_j corresponds to a pixel in the image and represents its intensity

$x_j = 0 \rightarrow$ black pixel

$x_j = 1 \rightarrow$ white pixel

Using a 2-layer NN

Consider 200 hidden units

$$\underline{H} = \sigma \left(\underbrace{\underline{X}}_{60000 \times 784} \underbrace{\underline{W}^{(1)T}}_{784 \times 200} + \underbrace{\underline{b}^{(1)T}}_{1 \times 200} \right)$$

$\uparrow$
60000 x 784

$$\underline{\hat{y}} = \underline{H} \underbrace{\underline{W}^{(2)T}}_{200 \times 10} + \underbrace{\underline{b}^{(2)T}}_{1 \times 10}$$

$\underline{H}$ is 60000×10

Backpropagation for training Neural Networks

- A neural network is a parametric model
- We will tune its parameters using optimization techniques such as SGD, ADAM, etc.
- To find suitable values for the parameters $\underline{\Theta}$, we will solve the optimization problem:

$$\hat{\underline{\Theta}} = \underset{\underline{\Theta}}{\operatorname{argmin}} \underbrace{J(\underline{\Theta})}_{\text{cost function}}$$

where

$$J(\underline{\Theta}) = \frac{1}{N} \sum_{i=1}^N \underbrace{L(y_i, f(x_i; \underline{\Theta}))}_{\substack{\text{loss function} \\ \text{for data point } (x_i, y_i)}}$$

$$\underline{\Theta} = \begin{bmatrix} \operatorname{vec}(\underline{W}^{(1)}) \\ b^{(1)} \\ \vdots \\ \operatorname{vec}(\underline{W}^{(L)}) \\ b^{(L)} \end{bmatrix}$$

Training Neural Network

- The functional form of the loss function depends on the problem

- Regression problems typically use squared error loss

$$L(y, f(\underline{x}; \underline{\theta})) = \left(y - \underbrace{f(\underline{x}; \underline{\theta})}_{\text{output of a neural net}} \right)^2$$

output of a neural net

- Multi-class classification problems typically use cross-entropy loss
(with M classes)

$$\begin{aligned} L(y, f(\underline{x}; \underline{\theta})) &= - \ln \underbrace{g_m \left(\underbrace{f(\underline{x}; \underline{\theta})}_{\text{is a vector of } M \times 1} \right)}_{\text{softmax}} \\ &= - \ln g_y \left(f(\underline{x}; \underline{\theta}) \right) \end{aligned}$$

we use training data label y as an index variable to select the correct logit

- These optimization problems cannot be solved in closed form

→ Numerical optimization algorithms have to be used

Training Neural Networks

- Numerical optimization update parameters in an iterative manner

In deep learning, one typically uses gradient-descent algorithms

Step 1: Pick an initial guess $\underline{\Theta}^{(0)}$

Step 2: Calculate gradient of the cost function w.r.t $\underline{\Theta}^{(t)}$, $t=0,1,2,\dots$

Step 3: Update the parameters as $\underline{\Theta}^{(t+1)} \leftarrow \underline{\Theta}^{(t)} - \gamma \nabla_{\underline{\Theta}} J(\underline{\Theta}^{(t)})$

Step 4: Terminate when some criterion is fulfilled, and take the last $\underline{\Theta}^{(t)}$ as $\hat{\underline{\Theta}}$

Training Neural Networks

Computational Challenges

1> Large datasets (N very large)

- The number of datapoints N is large in deep learning applications
- Makes computation of the cost function gradient very costly, due to the sum
- We resort to using a random subset of data to update parameters

↳ minibatch gradient descent

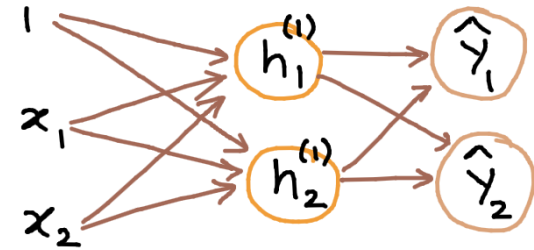
2> Large number of parameters $\underline{\theta}$

- The dimension of the parameter vector $\underline{\theta}$ is very large in deep learning
- To efficiently calculate the gradient $\nabla_{\underline{\theta}} J(\underline{\theta}^{(t)})$,
we need to apply chain rule of calculus

this will be done using the Backpropagation algorithm

Backpropagation

Neural net with 1-hidden layer
(with multiple outputs)



Forward pass

(to compute loss)

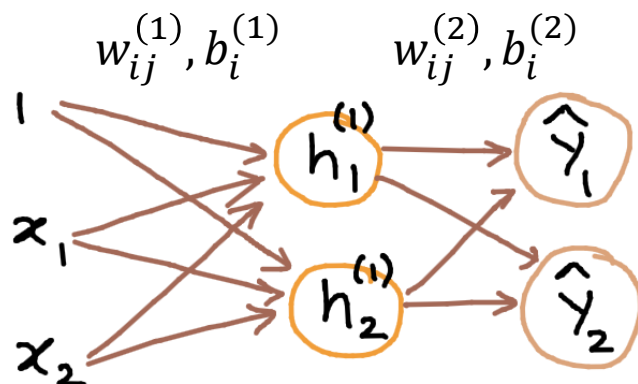
$$z_i^{(1)} = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

$$h_i^{(1)} = \sigma(z_i^{(1)})$$

$$\hat{y}_k = \sum_i w_{ki}^{(2)} h_i^{(1)} + b_k^{(2)}$$

$$L = \sum_k (\gamma_k - \hat{y}_k)^2$$

Backpropagation – contd.



Forward pass
(to compute loss)

$$z_i^{(1)} = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

$$h_i^{(1)} = \sigma(z_i^{(1)})$$

$$\hat{y}_k = \sum_i w_{ki}^{(2)} h_i^{(1)} + b_k^{(2)}$$

$$L = \sum_k (y_k - \hat{y}_k)^2$$

Backward pass
(to compute gradients)

$$\frac{\partial L}{\partial \hat{y}_k} = -2(y_k - \hat{y}_k)$$

$$1. \quad \frac{\partial L}{\partial w_{ki}^{(2)}} = \frac{\partial L}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial w_{ki}^{(2)}} = -2(y_k - \hat{y}_k) h_i^{(1)}$$

$$2. \quad \frac{\partial L}{\partial b_k^{(2)}} = \frac{\partial L}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial b_k^{(2)}} = -2(y_k - \hat{y}_k)$$

$$\frac{\partial L}{\partial h_i^{(1)}} = \sum_k \frac{\partial L}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial h_i^{(1)}} = \sum_k [-2(y_k - \hat{y}_k) w_{ki}^{(2)}]$$

$$\frac{\partial h_i^{(1)}}{\partial z_i^{(1)}} = \sigma'(z_i^{(1)})$$

$$3. \quad \frac{\partial L}{\partial w_{ij}^{(1)}} = \frac{\partial L}{\partial h_i^{(1)}} \frac{\partial h_i^{(1)}}{\partial z_i^{(1)}} \frac{\partial z_i^{(1)}}{\partial w_{ij}^{(1)}} = \frac{\partial L}{\partial h_i^{(1)}} \sigma'(z_i^{(1)}) x_j$$

$$4. \quad \frac{\partial L}{\partial b_i^{(1)}} = \frac{\partial L}{\partial h_i^{(1)}} \frac{\partial h_i^{(1)}}{\partial z_i^{(1)}} \frac{\partial z_i^{(1)}}{\partial b_i^{(1)}} = \frac{\partial L}{\partial h_i^{(1)}} \sigma'(z_i^{(1)})$$

Introduction to basic parametric models

- Backprop is based on the computational graph, and it basically works **backwards** through the graph, applying the chain rule at each node
- Backprop is used to train most neural nets you will find these days
- Even optimization algorithms much fancier than gradient descent (such as second-order methods) use backprop to compute gradients
- Once the derivatives w.r.t. the **weights** and **biases** are computed using backprop, the updates are applied to the weights and biases using some optimization scheme

$$\underline{\underline{w}}^{(t+1)} \leftarrow \underline{\underline{w}}^{(t)} - \eta \left. \frac{\partial J}{\partial \underline{\underline{w}}} \right|_{\underline{\underline{w}}^{(t)}} \qquad \underline{b}^{(t+1)} \leftarrow \underline{b}^{(t)} - \eta \left. \frac{\partial J}{\partial \underline{b}} \right|_{\underline{b}^{(t)}}$$

- Hand-calculation of derivatives are replaced with **automatic differentiation**